

Development, Simulation, and Testing of an Infrared-Based Wall-Following Controller

Laura Ferrari

*Department of Electrical and Computer Engineering
Florida Atlantic University
Boca Raton, FL, USA
lferrari2023@fau.edu*

Jordan Perrone

*Department of Electrical and Computer Engineering
Florida Atlantic University
Boca Raton, FL, USA
jperrone2024@fau.edu*

Abstract—This project investigates the design, simulation, and deployment of a wall-following behavior for the TurtleBot 4 mobile robot using ROS 2. The initial design utilized LiDAR-based perception and was tested in the Gazebo simulator; however, hardware-level communication constraints prevented reliable access to the robot's `/scan` topic during real-world testing. In response, the perception subsystem was redesigned to use the robot's on-board infrared (IR) proximity sensors, requiring a complete revision of the perception pipeline, message dependencies, and quality-of-service (QoS) configurations. The final system successfully performed stable wall-following behavior using a combination of IR intensity readings and a PID-based controller. Results demonstrate the feasibility of low-level IR-based perception for reactive navigation tasks and highlight practical lessons for transitioning from simulation to physical deployment in ROS 2-based mobile robot systems. The project source code is available at https://github.com/jperrone27/turtlebot4_wall_following.

Index Terms—TurtleBot 4, ROS 2, wall following, infrared sensing, mobile robotics, Gazebo

I. INTRODUCTION

Wall following is a foundational capability in mobile robotics relevant for numerous higher-level behaviors including corridor navigation, obstacle avoidance, and exploration. The algorithmic logic for wall following is conceptually straightforward, yet its implementation on real hardware exposes many subtleties in sensing, control, and software integration.

The primary focus of this effort was to gain practical experience in ROS 2 node design, sensor data processing, and overcoming real-world control challenges. For modern educational and research platforms such as the TurtleBot 4, wall following provides an ideal context for studying closed-loop control, sensor integration, and the challenges of transferring simulated robotic behaviors to physical hardware.

The goal of this project was to develop a modular ROS 2 application capable of performing wall following using perception and control nodes operating within a well-structured package architecture. Limitations with the processing and communication of LiDAR information required a redesign of the perception subsystem to rely on the TurtleBot 4's on-board IR sensors instead of LiDAR. In order to document results, odometry logging was implemented to plot the 2-D path of the

robot during execution of the wall-following task, providing numerical results in addition to video recordings.

This report documents the problem formulation, design, implementation, and results of an IR-based ROS 2 package architecture designed to induce wall-tracking behavior, along with critical debugging lessons learned regarding QoS and data logging.

II. PROBLEM FORMULATION

A. Problem Definition

The wall-following task requires the robot to maintain a desired lateral distance from a wall while moving forward. Achieving this behavior depends on continuous sensory feedback and an effective control law capable of regulating the distance error. Initially, this problem was framed in terms of LiDAR-based perception, where distance to a wall could be extracted through geometric processing of range data. However, due to real-world LiDAR data collection limitations discussed in the following sections, the proposed wall-following solution was redesigned using IR sensor intensity measurements. IR sensors provide a monotonic but not directly metric indication of proximity.

B. Experimental Apparatus and Setup

1) *Hardware Configuration:* The wall-following controller was developed and deployed on the iRobot Create[®] 3 educational robot integrated into the TurtleBot 4 platform. This system is designed as a modern, ROS 2-native mobile robot testbed [1].

The core platform, the iRobot Create 3, is based on the design of the Roomba[®] robot vacuum, providing a compact and highly mobile research platform capable of navigating and mapping internal spaces. Key features relevant to this project include a comprehensive array of built-in sensors. The front bumper assembly hosts seven pairs of IR proximity sensors across multiple zones, which were leveraged for the final wall-following controller. The base is also equipped with four cliff sensors, two drive wheels with encoders and current sensors, and an optical odometry sensor. Internally, an inertial measurement unit (IMU) fuses with odometry sensors to generate a robust odometry estimate, critical for the path logging implemented in this work. The robot's faceplate

and cargo bay feature a modular hole pattern and cable passthroughs, supporting quick removal for prototyping and payload integration, though the sensor-actuator system was primarily utilized in its base configuration.

2) *Development Environment and System Validation:* The development workflow utilized a standard ROS 2 remote control setup. All packages were developed on a dedicated Ubuntu PC, which connected to the robot's Raspberry Pi 5 over a shared local Wi-Fi network. This configuration enabled remote ROS 2 operations, including node launching, topic subscription, and control execution from the host PC's Bash terminal. To facilitate secure and flexible development independent of local network restrictions, Visual Studio Code (VS Code) Secure Tunneling was configured.

Prior to software development, the system was validated to ensure operational readiness. This included reviewing the iRobot Create 3 and ROS 2 Humble documentation [2], properly charging and starting the TurtleBot 4, and confirming that standard ROS 2 topics (e.g., publishing to `/cmd_vel` for motion, echoing sensor data) were functional for basic remote control and communication.

3) *Simulation Environment Setup:* Initial development and testing of the wall-following behavior, specifically for the LiDAR-based perception pipeline, were conducted in the Gazebo simulation environment. This required the installation of a Windows Subsystem for Linux (WSL) Ubuntu distribution on the host PC to correctly configure the Gazebo environment and manage the ROS 2 workspace. The foundational ROS 2 packages, originally designed for wall perception using simulated LiDAR data, were built and refined within this environment. Confirmation of stable wall tracking in the idealized Gazebo simulation marked the end of the initial design phase before deployment to the physical TurtleBot 4 hardware.

C. Inputs

1) *Simulation Input (Gazebo):*

- `/scan` — LiDAR range data providing angular observations of nearby obstacles.

2) *Real Robot Input (TurtleBot 4):*

- `/ir_intensity` — `irobot_create_msgs/msg/IrIntensityVector`, consisting of intensity values from multiple IR proximity sensors.

D. Outputs

- `/wall_tracking` — A custom message containing the computed distance error and angular error.
- `/cmd_vel` — Linear and angular velocity commands generated by the controller to achieve wall following.
- `/wf_odometry` — Timestamped (x, y) position coordinates saved to a local CSV log file for offline plotting and path visualization.

E. Constraints and Assumptions

Real-time LiDAR data was unavailable, as the `/scan` topic was found to be unstable or unavailable on the Ubuntu remote

control PC configured to communicate with the Raspberry Pi onboard the TurtleBot. The issue is likely due to communication challenges between the remote PC and the robot's Raspberry Pi, as the `/scan` topic was found to function properly on the Raspberry Pi. Thus, IR-based sensing was adopted as the primary modality.

Raw IR intensity values are not a calibrated measure of distance. As such, the control node processes relative intensities from each IR sensor and compares them to an empirically derived target intensity, which represents the desired stand-off distance.

The following assumptions were made as part of the experimental design:

- IR intensity increases monotonically with proximity.
- A stable target intensity can be empirically determined.
- The robot operates at low forward speeds to maintain stability.
- The robot is not required to avoid or track obstacles of significantly different geometry compared to a typical wall.

III. METHODS & DEVELOPMENT

A. Method

Wall-following methods have traditionally been grouped into two main categories in robotics research:

- 1) **Reactive Methods:** These approaches use immediate sensor feedback (e.g., IR or ultrasonic sensing) to adjust robot orientation relative to a wall. Their simplicity affords real-time operation but limits robustness under complex geometries.
- 2) **Geometry-Based Methods Using LiDAR:** Common in research robotics, LiDAR-based methods apply techniques such as line fitting, RANSAC, or minimum-range extraction to infer wall orientation and distance. These methods offer improved interpretability but require heavier computation and reliable sensor streams.

This project initially followed the second category (geometry-based LiDAR processing) but ultimately implemented a reactive IR-based method due to hardware constraints. This strategic change reflects a common theme in robotics research: the viability of a robust solution is often predicated more on reliable sensor availability than on algorithmic complexity.

B. Robotic Control Software Design Overview

1) *ROS 2 Packages:* The system was organized into Python-based ROS 2 packages (`ament_python`):

- 1) **wall_following_perception** — Implements perception nodes for computing wall-tracking error.
- 2) **wall_following_controller** — Implements PID-based control of linear and angular velocities.
- 3) **wall_following_bringup** — Contains launch files integrating the perception and controller nodes.
- 4) **wf_odometry** — Contains a node for subscribing to `/odom` and safely logging path data to disk.

This modular design facilitates isolated development, testing, and debugging while promoting reusability.

2) *Initial LiDAR-Based Design (Simulation Phase)*: The first implementation relied on LiDAR data from the `/scan` topic in the Gazebo simulation environment. In this setup, the TurtleBot 4's simulated 2D LiDAR provided dense and highly stable range measurements at a fixed angular resolution. The perception logic began with angular filtering to isolate the sectors of the scan corresponding to the robot's left or right side, depending on which wall was being followed. From these filtered ranges, geometric processing techniques were applied to compute the robot's lateral distance to the wall. Wall orientation was estimated by comparing the distances at two separated angular indices, which produced an angular error signal useful for aligning the robot parallel to the wall. These computed quantities were then packaged into a custom tracking-error message and published for consumption by the controller node. Simulated Gazebo testing validated this LiDAR-based pipeline, demonstrating that under idealized conditions—noise-free sensing, perfect ground truth physics, and uninterrupted communication—the controller could maintain a stable and consistent offset from the wall. The robot exhibited smooth corrections in both distance and heading, confirming that the underlying geometric method was theoretically sound. However, this design never reached real-world viability due to the inability to access consistent `/scan` data from the robot's hardware.

3) *Design Revision: Transition from LiDAR to IR Sensors*: The proposed perception and control algorithms underwent a substantive redesign upon discovery of communication instability of LiDAR data between the Raspberry Pi controller on the TurtleBot 4 and the Ubuntu remote PC. The troubleshooting process lasted an entire working day, but no solution was found to allow LiDAR-based perception and control. The two computing devices were preconfigured to communicate data over the Wi-Fi network, and communication system design is beyond the scope of this entry-level project.

As such, the revision process included:

- 1) **Sensor Selection**: Identified the `/ir_intensity` topic as reliable and sufficiently frequent for reactive control.
- 2) **Message Inspection**: Used `ros2 topic echo /ir_intensity` to determine the correct sensor names contained within `header.frame_id` for indexing specific IR readings.
- 3) **Perception Logic**: Distance error was revised based on a single calibrated IR sensor and logic was designed for linear and angular velocity to ensure wall-tracking behavior along nonlinear sections of walls.
- 4) **Dependency Updates**: Added `irobot_create_msgs` to the perception package's `package.xml`.
- 5) **QoS Compatibility Fix**: IR data uses a sensor-data QoS profile. Subscribing with the default QoS triggered incompatible QoS errors; thus the perception node explicitly subscribed using `qos_profile_sensor_data`.

4) *Build and Launch Integration*: After revising the nodes, the entire workspace was rebuilt using `colcon build --symlink-install` in the Bash terminal. Environment variables were properly sourced using `source install/setup.bash`. The launch file in the `bringup` package orchestrated both nodes, ensuring correct namespace and topic interconnection.

C. Program Logic Design

The wall-following program is a modular ROS 2 application composed of three primary packages — `wall_following_perception`, `wall_following_controller`, and `wf_odometry` — orchestrated by a unified launch system in the `wf_bringup` package. This architecture adheres to the software design principle of separation of concerns: the perception node processes raw sensor data, the controller node applies the control law, and the odometry node handles data logging.

The central execution flow begins with the *WallPerceptionNode*, which subscribes to the `/ir_intensity` topic using the critical `qos_profile_sensor_data` to ensure reliable reception of high-frequency IR sensor data. This node's core logic involves an initial phase where it determines the target wall side ("left" or "right") by comparing the aggregated intensity readings of sensor clusters. Once the side is determined, it calculates two primary error signals. The distance error is computed as the difference between an empirically determined `target_intensity` (representing the desired standoff distance) and the observed intensity of the rear-side IR sensor proxy. The angular error is calculated as the difference between the front-side and rear-side IR sensor proxies. These three values (distance error, angular error, and a side flag) are published as a `Float64MultiArray` to the `/wall_tracking` topic.

The *WallControllerNode* subscribes to `/wall_tracking` and implements the control law. It utilizes the received distance and angular errors to compute the required angular velocity ω command using a PD-like structure:

$$\omega = (K_d \cdot \text{distance_error} + K_\theta \cdot \text{angle_error}) \cdot (-\text{side_flag}),$$

where `side_flag` ensures the corrective turn is always directed towards the wall. The linear velocity v is maintained at a nominal, low-speed value, with a suppression factor applied during aggressive turns (high ω) to maintain stability. The resulting linear and angular velocities are encapsulated in a `geometry_msgs/msg/Twist` message and published to the `/cmd_vel` topic to drive the robot.

Finally, the *OdometryLoggerNode* subscribes to the high-frequency `/odom` topic using the `BEST_EFFORT` QoS profile and safely logs the robot's (x, y) position coordinates to a timestamped CSV file. The file is opened in unbuffered binary mode to guarantee data integrity and persistence even in the event of an unexpected node termination.

IV. RESULTS AND DISCUSSION

A. Simulation Results

The initial design, which relied on the geometric processing of simulated LiDAR data, performed predictably and successfully within the idealized Gazebo environment. The robot consistently achieved and maintained a near-constant offset from the simulated wall. The controller utilized the highly stable and precise range information from the `/scan` topic, allowing the control loop to maintain smooth and robust response characteristics with minimal positional oscillation. These simulation results provided a baseline confirmation of the control law's viability under perfect sensor conditions, demonstrating the theoretical success of a geometry-based wall-following approach.

B. Real-World Results Using IR Sensors

The transition from simulation to the physical TurtleBot 4 hardware necessitated a significant shift in sensing modality and a subsequent debugging effort due to hardware constraints. The primary challenge was the unreliable availability of the `/scan` topic on the remote control PC, forcing a pivot to the low-level IR proximity sensors. This practical pivot exposed several key ROS 2 and platform-specific integration challenges:

- **QoS Mismatch Resolution:** A critical initial failure was traced to a QoS mismatch. The perception node, subscribing with the default Reliable QoS profile, could not establish a connection with the `/ir_intensity` topic, which publishes sensor data using the Best Effort profile. This was resolved by explicitly configuring the perception node's subscription to use `qos_profile_sensor_data`, underscoring the necessity of matching the QoS contract between publishers and subscribers in production ROS 2 systems.
- **Sensor Frame ID Resolution:** Warnings regarding unreadable sensor frames (e.g., "could not find reading for `ir_right`") were resolved by inspecting the raw `IrIntensityVector` message structure. This revealed the required frame identifiers (e.g., `ir_intensity_right`), which were subsequently incorporated into the perception logic to correctly index the required sensor readings.
- **Empirical Target Selection:** Since IR intensity is non-metric, a target intensity value, representing the desired lateral standoff distance, had to be empirically determined through physical testing and calibration. This value was then parameterized in the `WallPerceptionNode`.
- **Odometry Logging Reliability:** The `OdometryLoggerNode` was developed to provide quantitative performance evaluation. To counter file buffering issues in the high-frequency logging of the `/odom` topic, the CSV file was opened using unbuffered, binary I/O. Furthermore, data strings were manually encoded prior to writing, guaranteeing that odometry data persisted immediately to disk, preventing data loss upon unexpected node termination.

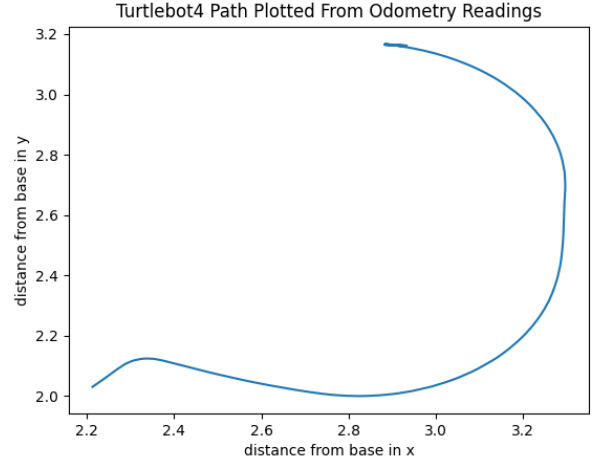


Fig. 1. 2-D path plotted from odometry sensor readings (units in meters).

Following these revisions, the final system successfully performed stable lateral wall tracking. Fig. 1 shows the 2-D path plotted from odometry data logged during testing, where the units are in meters and measured with respect to the robot's charging base (the origin). The plot begins at the starting location for the robot, which was positioned adjacent (but not parallel) to a cubicle wall. The robot detects the nearest wall and navigates along it into the next adjacent cubicle. A large box in the adjacent cubicle acts as a wall due to its height and minimally reflective cardboard material. The robot navigates along the box and turns the corner around it to follow the perpendicular side. A video demonstration of this test can be accessed via the following public YouTube link: <https://youtube.com/shorts/YgjRR1WlokQ?feature=share>.

C. Controller Tuning

The controller utilized a proportional-derivative (PD) structure for reactive control, which was tuned following standard robotics practice [3]. The tuning began with proportional-only control on the distance error to establish responsiveness. A small derivative term was then introduced to dampen minor fluctuations caused by inherent IR sensor noise. Crucially, due to the switch to single-sensor IR distance measurement, the angular alignment term K_θ was set to zero, simplifying the controller to a distance-error-only regulator. By maintaining a low, constant forward velocity and managing the maximum angular velocity, the system achieved stable tracking behavior, confirming that even coarse, low-level IR sensing is sufficient for fundamental reactive navigation tasks like wall following.

D. Discussion

The development process revealed several practical insights critical to real-world mobile robotics implementation. The failure of the LiDAR-based design in the physical world, despite its success in simulation, highlights the significant gap between idealized models and real-world hardware, necessi-

tating flexibility and preparedness for fundamental redesigns based on hardware constraints.

Furthermore, the QoS mismatch was a fundamental communication inhibitor. This experience underscores the critical necessity of understanding and explicitly configuring QoS profiles in ROS 2 systems, particularly when integrating sensor data (Best Effort) with control components (Reliable). Additionally, the specific solution required to ensure the persistence of high-frequency odometry data (unbuffered binary file I/O) is a practical lesson in data integrity and I/O management within time-critical Python-based ROS 2 nodes.

The modular project design—separating perception, control, and logging packages—allowed for organized and efficient development, and is important for further extension and integration. This entry-level project provided key insights into the robotic software development and testing process and demonstrated effective implementation of the desired wall-tracking behavior with the available hardware configuration.

E. Limitations and Future Directions

The final IR-based solution inherently imposed several limitations: IR sensors are non-metric and susceptible to environmental factors like ambient light and surface reflectivity. Furthermore, relying on a single lateral sensor proxy eliminated the capability for angular alignment, which is generally achieved using multi-point geometric inference. Future work should focus on addressing these limitations:

- **Multi-Sensor Fusion:** Investigate fusing intensity readings from multiple IR sensors to derive a coarse approximation of the wall’s angular alignment, allowing the full PD control structure to be utilized.
- **LiDAR Integration:** Reattempt integration of the LiDAR-based pipeline if network or hardware reliability can be improved, leveraging the benefits of metric distance and geometric alignment.
- **Sensor Filtering:** Incorporate a low-pass filter within the perception node to smooth IR noise, potentially allowing higher forward speeds without compromising stability.
- **Higher-Level Integration:** Integrate the tested wall-following behavior with complex behaviors such as simultaneous localization and mapping (SLAM) or higher-level navigation stacks.
- **Simulation Consistency:** Revise simulations to better mirror real-world implementations, enabling more useful data collection to guide future development.

V. CONCLUSIONS

The goal of this project was to develop a stable, modular ROS 2 application for wall following on the TurtleBot 4, providing practical experience in closed-loop control and system integration. While the initial objective focused on a geometrically sophisticated, LiDAR-based solution validated in Gazebo, real-world communication constraints forced a critical and successful pivot to a reactive approach utilizing

the robot’s onboard IR proximity sensors. This adaptive redesign became a key contribution of this work, highlighting fundamental lessons in robotics system development.

The final implemented system, orchestrated through the `wf_bringup` package, successfully demonstrated stable wall-following behavior. This stability was achieved by decoupling the application into three isolated and specialized packages: the `WallPerceptionNode` utilized aggregated IR intensity readings to compute stand-off and alignment error proxies, which were then consumed by the `WallControllerNode` running a proportional control law. The modular architecture proved invaluable, allowing the sensing modality to be fundamentally changed without requiring modification to the core controller logic.

Key practical findings emerged from the deployment process. One such finding was the critical importance of QoS management in ROS 2, where a mismatch between the sensor’s Best Effort profile and the default Reliable profile can prevent data flow entirely. A second was the necessity of ensuring real-time data integrity by managing file buffering during high-frequency odometry logging. These lessons underscore that successful transition from simulation to physical hardware requires robust low-level engineering practices in addition to algorithmic correctness. In conclusion, this project validates the feasibility of using low-cost, reactive IR sensing for fundamental mobile robot navigation and provides a case study in developing and debugging ROS 2 applications under real-world hardware limitations.

APPENDIX: TEAM COLLABORATIONS

Although the project was conducted collaboratively, each member contributed uniquely and substantially. The following list summarizes each member’s contributions.

Laura Ferrari

- Installed Windows Subsystem for Linux (Ubuntu) distribution.
- Learned Linux and Bash scripting fundamentals and best practices.
- Configured the Gazebo environment for simulation.
- Developed initial Python packages for LiDAR-based wall perception and tracking.
- Refined logic and confirmed simulation results for wall tracking in Gazebo.
- Configured the Ubuntu remote control PC and TurtleBot 4 on a shared Wi-Fi network.
- Performed the initial attempt of wall-following demonstration.
- Tested and troubleshot LiDAR sensor issues.
- Led development and refinement of the presentation.
- Collaborated in development of the final report.

Jordan Perrone

- Researched and studied documentation for iRobot Create 3 and ROS 2 Humble.

- Configured the Ubuntu remote control PC and TurtleBot 4 on a shared Wi-Fi network.
- Ensured proper battery charging and startup of TurtleBot 4; tested preconfigured/standard ROS 2 topics (e.g., Twist commands, topic echo).
- Built and installed packages originally designed for simulation with LiDAR for wall perception.
- Tested and troubleshoot LiDAR sensor issues.
- Tested IR sensor readings and redesigned ROS 2 packages for IR perception to replace LiDAR perception and control logic.
- Tested and refined the IR-based tracking system and empirically derived a minimum IR intensity for the desired wall-tracking distance.
- Added a new package for logging odometry readings and plotted the 2-D path.
- Performed the final demonstration of wall-following.
- Collaborated in development of the final report.

REFERENCES

- [1] “Create® 3 Docs,” iRobot Education. [Online]. Available: https://iroboteducation.github.io/create3_docs/
- [2] “ROS 2 Documentation — Humble,” ROS 2 Docs. [Online]. Available: <https://docs.ros.org/en/humble/>
- [3] J. J. Craig, *Introduction to Robotics: Mechanics and Control*, 3rd ed. Harlow, U.K.: Pearson Education, 2013.